

Database System Engineering and Distributed Backend Development

JOINS AND SET OPERATIONS

Name: Hansika

Roll No: 2520030237

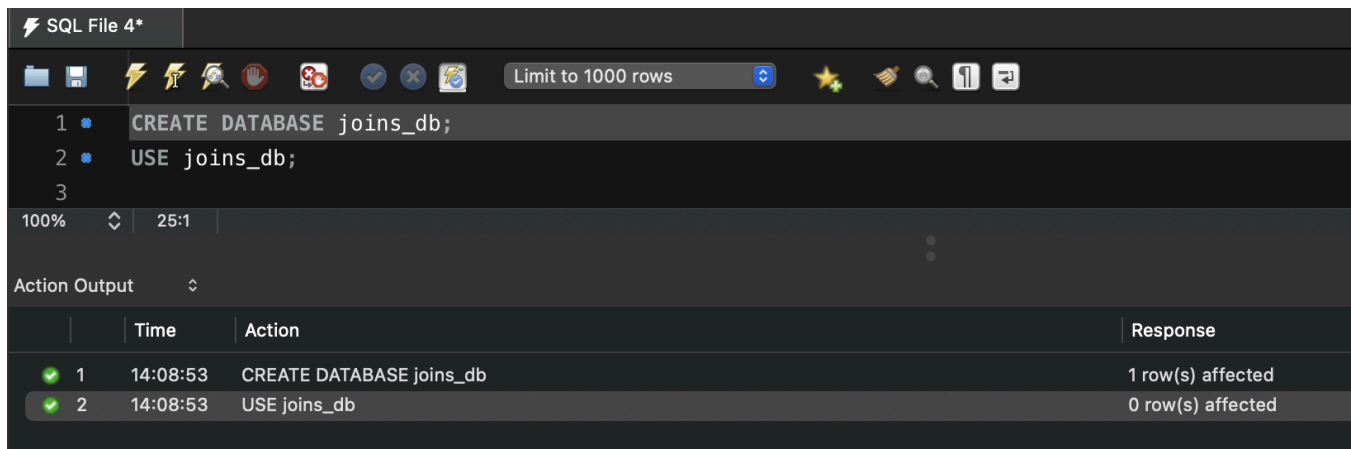
Section: 7

Email: gunapatihansikareddy@klh.edu.in

Step 1 — Create and Select a Database

Query:

```
CREATE DATABASE joins_db;
USE joins_db;
```



Step 2 — Create the Tables (class, class_info)

Query:

```
CREATE TABLE class (
    id INT,
    name VARCHAR(30)
);

CREATE TABLE class_info (
    id INT,
    address VARCHAR(30)
);
```

```

3 CREATE TABLE class (
4     id INT,
5     name VARCHAR(30)
6 );
7
8 CREATE TABLE class_info (
9     id INT,
10    address VARCHAR(30)
11 );
12
13

```

100% 1:13

Action Output

	Time	Action
1	14:08:53	CREATE DATABASE joins_db
2	14:08:53	USE joins_db
3	14:10:14	CREATE TABLE class (id INT, name VARCHAR(30))
4	14:10:14	CREATE TABLE class_info (id INT, address VARCHAR(30))

Step 3 — Insert Data

Query:

```

INSERT INTO class VALUES
(1,'Hansika'),
(2,'Hasini'),
(4,'Bindhu');

INSERT INTO class_info VALUES
(1,'DELHI'),
(2,'MUMBAI'),
(3,'CHENNAI');

```

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and Performance
- Data Export
- Data Import

Object Inspector: No object selected

```

11 );
12 INSERT INTO class VALUES
13 (1,'Hansika'),
14 (2,'Hasini'),
15 (4,'Bindhu');
16
17 INSERT INTO class_info VALUES
18 (1,'DELHI'),
19 (2,'MUMBAI'),
20 (3,'CHENNAI');
21
22

```

100% 15:20

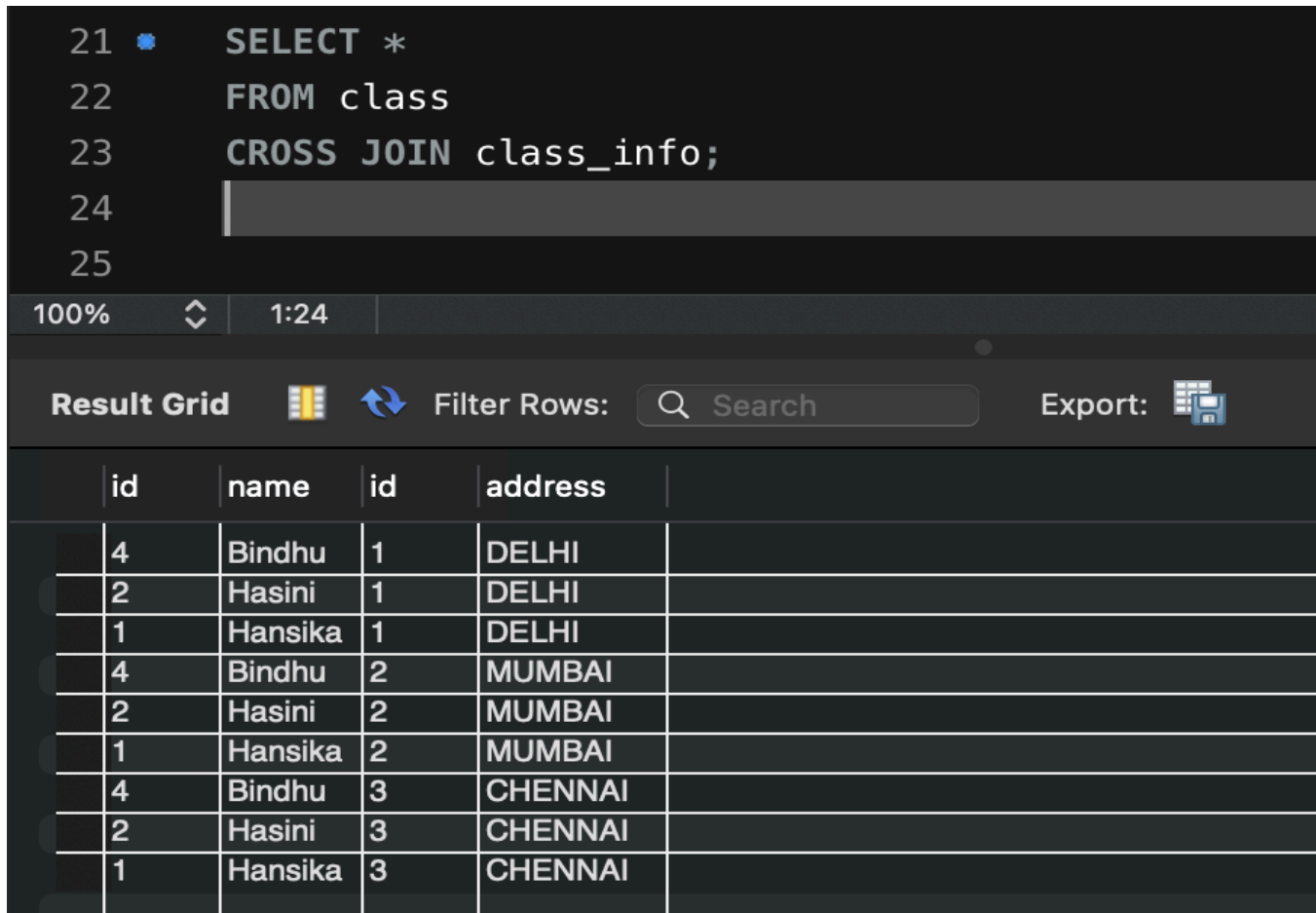
Action Output

	Time	Action	Response
1	14:08:53	CREATE DATA...	1 row(s) affected
2	14:08:53	USE joins_db	0 row(s) affected
3	14:10:14	CREATE TABL...	0 row(s) affected
4	14:10:14	CREATE TABL...	0 row(s) affected
5	10:22:46	INSERT INTO...	3 row(s) affected Records: 3 Duplicates: 0 Warnings...
6	10:22:46	INSERT INTO...	3 row(s) affected Records: 3 Duplicates: 0 Warnings...

Step 4 — Question 1: CROSS JOIN

Query:

```
SELECT *  
FROM class  
CROSS JOIN class_info;
```



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
21 SELECT *  
22 FROM class  
23 CROSS JOIN class_info;  
24  
25
```

Below the query editor, the 'Result Grid' is displayed. It shows the output of the CROSS JOIN query. The grid has 5 columns: 'id', 'name', 'id', 'address', and an empty column. The data is as follows:

	id	name	id	address	
	4	Bindhu	1	DELHI	
	2	Hasini	1	DELHI	
	1	Hansika	1	DELHI	
	4	Bindhu	2	MUMBAI	
	2	Hasini	2	MUMBAI	
	1	Hansika	2	MUMBAI	
	4	Bindhu	3	CHENNAI	
	2	Hasini	3	CHENNAI	
	1	Hansika	3	CHENNAI	

Step 5 — Recreate Tables for INNER JOIN Section

Query:

```
DROP TABLE class, class_info;  
  
CREATE TABLE class (  
    id INT,  
    name VARCHAR(30)  
);  
  
CREATE TABLE class_info (  
    id INT,  
    address VARCHAR(30)  
);
```

```
24 DROP TABLE class, class_info;
25
26 CREATE TABLE class (
27     id INT,
28     name VARCHAR(30)
29 );
30
31 CREATE TABLE class_info (
32     id INT,
33     address VARCHAR(30)
34 );
35
36
37
```

00% 1:35

Step 6 — Insert Updated Data

Query:

```
INSERT INTO class VALUES
(1,'Hansika'),
(2,'Hasini'),
(3,'Bindhu'),
(4,'Anusri');

INSERT INTO class_info VALUES
(1,'DELHI'),
(2,'MUMBAI'),
(3,'CHENNAI');
```

```

35 • INSERT INTO class VALUES
36     (1, 'Hansika'),
37     (2, 'Hasini'),
38     (3, 'Bindhu'),
39     (4, 'Anusri');
40
41 • INSERT INTO class_info VALUES
42     (1, 'DELHI'),
43     (2, 'MUMBAI'),
44     (3, 'CHENNAI');

```

100% 12:32

Step 7 — Question 2: INNER JOIN

Query:

```

SELECT *
FROM class
INNER JOIN class_info
ON class.id = class_info.id;

```

Data Import

Object I... Session

No object
selected

```

45
46 • SELECT *
47 FROM class
48 INNER JOIN class_info
49 ON class.id = class_info.id;

```

100% 14:43

Result Grid



Filter Rows:



Search

Export:



	id	name	id	address	
	1	Hansika	1	DELHI	
	1	Hansika	1	DELHI	
	2	Hasini	2	MUMBAI	
	2	Hasini	2	MUMBAI	
	3	Bindhu	3	CHENNAI	
	3	Bindhu	3	CHENNAI	
	1	Hansika	1	DELHI	
	1	Hansika	1	DELHI	
	2	Hasini	2	MUMBAI	
	2	Hasini	2	MUMBAI	
	3	Bindhu	3	CHENNAI	
	3	Bindhu	3	CHENNAI	

Step 8 — Question 3: INNER JOIN (Selected Columns)

Query:

```
SELECT class.name,  
       class_info.address  
FROM class  
INNER JOIN class_info  
ON class.id = class_info.id;
```

The screenshot shows a database IDE interface. On the left, there's a sidebar with icons for 'Status and', 'Data Export', and 'Data Import'. Below these, there's a section for 'Object Explorer' and 'Session' with the text 'No object selected'. The main area displays an SQL query in a dark-themed editor. The query is an INNER JOIN between 'class' and 'class_info' tables, selecting 'name' from 'class' and 'address' from 'class_info'. Below the query editor, there's a 'Result Grid' section. It includes a 'Filter Rows' search bar and an 'Export' button. The result grid shows a table with two columns: 'name' and 'address'. The data is as follows:

name	address
Hansika	DELHI
Hansika	DELHI
Hasini	MUMBAI
Hasini	MUMBAI
Bindhu	CHENNAI
Bindhu	CHENNAI
Hansika	DELHI
Hansika	DELHI
Hasini	MUMBAI
Hasini	MUMBAI
Bindhu	CHENNAI
Bindhu	CHENNAI

Step 9 — Question 4: NATURAL JOIN

Query:

```
SELECT *  
FROM class  
NATURAL JOIN class_info;
```

Data Export
Data Import

Object Explorer
Session
No object selected

```

56
57 SELECT *
58 FROM class
59 NATURAL JOIN class_info;
60
61

```

100% 1:61

Result Grid Filter Rows: Search Export:

	id	name	address
☐	1	Hansika	DELHI
☐	1	Hansika	DELHI
☐	2	Hasini	MUMBAI
☐	2	Hasini	MUMBAI
☐	3	Bindhu	CHENNAI
☐	3	Bindhu	CHENNAI
☐	1	Hansika	DELHI
☐	1	Hansika	DELHI
☐	2	Hasini	MUMBAI
☐	2	Hasini	MUMBAI
☐	3	Bindhu	CHENNAI
☐	3	Bindhu	CHENNAI

Step 10 — Insert Extra Rows for Outer Join Section

Query:

```

INSERT INTO class VALUES
(5, 'Rohan');

INSERT INTO class_info VALUES
(7, 'NOIDA'),
(8, 'PANIPAT');

```

```

61 INSERT INTO class VALUES
62     (5, 'Rohan');
63
64 INSERT INTO class_info VALUES
65     (7, 'NOIDA'),
66     (8, 'PANIPAT');
67

```

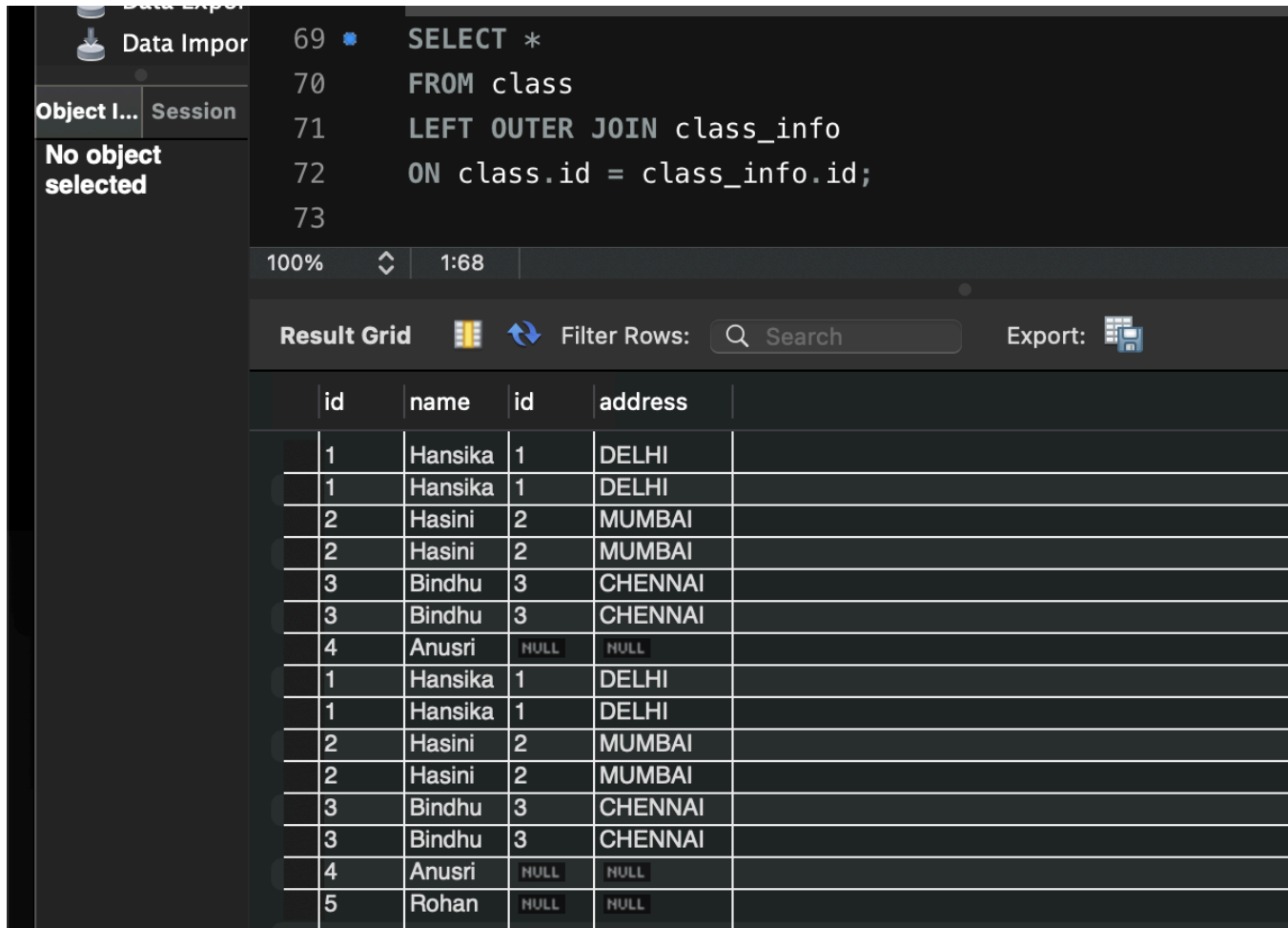
100% 1:56

Query Completed

Step 11 — Question 5: LEFT OUTER JOIN

Query:

```
SELECT *  
FROM class  
LEFT OUTER JOIN class_info  
ON class.id = class_info.id;
```



The screenshot shows a database IDE interface. On the left, there's a sidebar with 'Data Import' and 'Object Explorer' tabs. The 'Object Explorer' shows 'No object selected'. The main area displays a SQL query in a dark-themed editor. Below the query, there's a 'Result Grid' showing the output of the query. The query is a LEFT OUTER JOIN between 'class' and 'class_info' tables. The result grid shows 15 rows of data. The first 12 rows are matches from the 'class' table, and the last 3 rows are unmatched rows from the 'class' table where 'class_info.id' is NULL.

	id	name	id	address
	1	Hansika	1	DELHI
	1	Hansika	1	DELHI
	2	Hasini	2	MUMBAI
	2	Hasini	2	MUMBAI
	3	Bindhu	3	CHENNAI
	3	Bindhu	3	CHENNAI
	4	Anusri	NULL	NULL
	1	Hansika	1	DELHI
	1	Hansika	1	DELHI
	2	Hasini	2	MUMBAI
	2	Hasini	2	MUMBAI
	3	Bindhu	3	CHENNAI
	3	Bindhu	3	CHENNAI
	4	Anusri	NULL	NULL
	5	Rohan	NULL	NULL

Step 12 — Question 6: LEFT JOIN (Unmatched Only)

Query:

```
SELECT *  
FROM class  
LEFT JOIN class_info  
ON class.id = class_info.id  
WHERE class_info.id IS NULL;
```


Users and I
Status and
Data Expor
Data Impor

Object I... Session
No object selected

```

75 SELECT *
76 FROM class
77 LEFT JOIN class_info
78 ON class.id = class_info.id
79 WHERE class_info.id IS NULL;
80

```

100% 11:70

Result Grid Filter Rows: Search Export:

	id	name	id	address
	4	Anusri	NULL	NULL
	4	Anusri	NULL	NULL
	5	Rohan	NULL	NULL

Result Grid
Form Editor

Step 13 — Question 7: RIGHT OUTER JOIN

Query:

```

SELECT *
FROM class
RIGHT OUTER JOIN class_info
ON class.id = class_info.id;

```

```

82 SELECT *
83 FROM class
84 RIGHT OUTER JOIN class_info
85 ON class.id = class_info.id;
86

```

100% 1:81

Result Grid Filter Rows: Search Export:

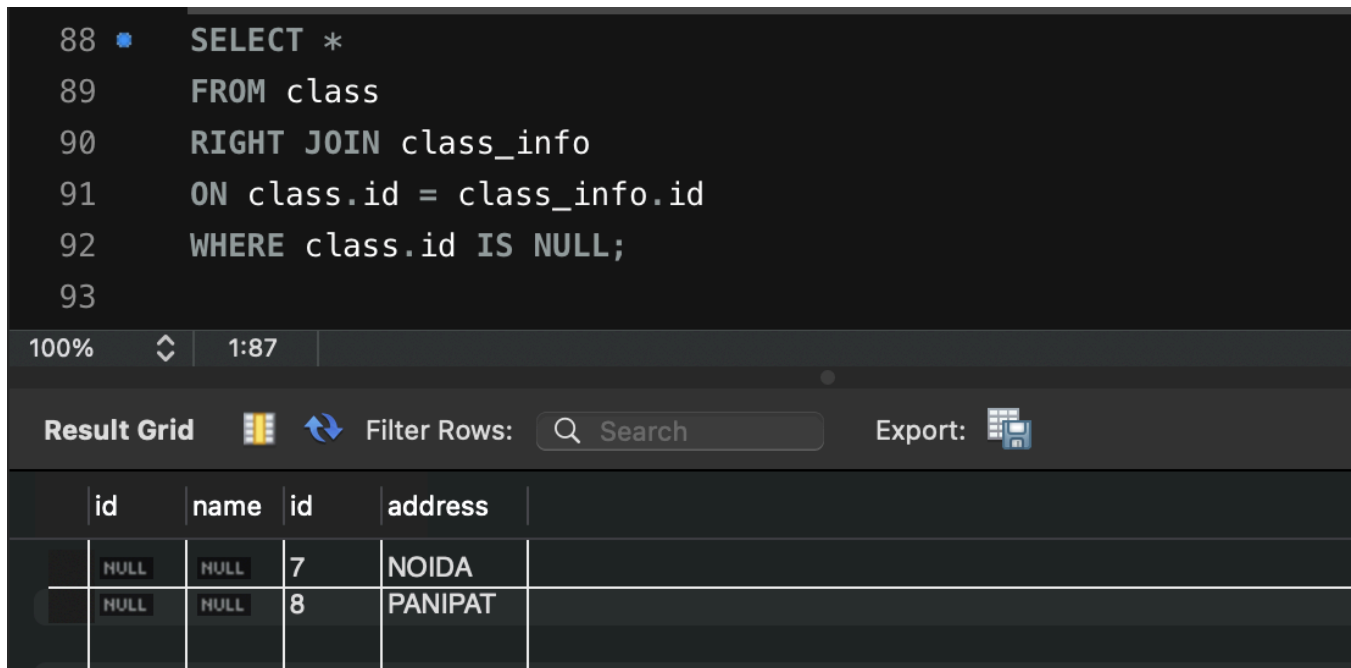
	id	name	id	address
	1	Hansika	1	DELHI
	1	Hansika	1	DELHI
	2	Hasini	2	MUMBAI
	2	Hasini	2	MUMBAI
	3	Bindhu	3	CHENNAI
	3	Bindhu	3	CHENNAI
	1	Hansika	1	DELHI
	1	Hansika	1	DELHI
	2	Hasini	2	MUMBAI
	2	Hasini	2	MUMBAI
	3	Bindhu	3	CHENNAI
	3	Bindhu	3	CHENNAI
	NULL	NULL	7	NOIDA
	NULL	NULL	8	PANIPAT

Result Grid
Form Editor
Field Types
Query Stats

Step 14 — Question 8: RIGHT JOIN (Unmatched Only)

Query:

```
SELECT *
FROM class
RIGHT JOIN class_info
ON class.id = class_info.id
WHERE class.id IS NULL;
```



The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
88 SELECT *
89 FROM class
90 RIGHT JOIN class_info
91 ON class.id = class_info.id
92 WHERE class.id IS NULL;
93
```

Below the query editor, the 'Result Grid' is displayed. It shows two rows of data. The first row has columns 'id', 'name', 'id', and 'address' with values 'NULL', 'NULL', '7', and 'NOIDA' respectively. The second row has values 'NULL', 'NULL', '8', and 'PANIPAT'.

	id	name	id	address
	NULL	NULL	7	NOIDA
	NULL	NULL	8	PANIPAT

Step 15 — Question 9: FULL OUTER JOIN

Query:

```
SELECT *
FROM class
LEFT JOIN class_info
ON class.id = class_info.id
UNION
SELECT *
FROM class
RIGHT JOIN class_info
ON class.id = class_info.id;
```

current caret position or to toggle automatic help.

```

94
95 SELECT *
96 FROM class
97 LEFT JOIN class_info
98 ON class.id = class_info.id
99 UNION
100 SELECT *
101 FROM class
102 RIGHT JOIN class_info
103 ON class.id = class_info.id;
104
100% 24:92

```

Result Grid

id	name	id	address
1	Hansika	1	DELHI
2	Hasini	2	MUMBAI
3	Bindhu	3	CHENNAI
4	Anusri	NULL	NULL
5	Rohan	NULL	NULL
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

Result Grid Form Editor

Step 16 — Question 10: FULL OUTER JOIN (Unmatched Only)

Query:

```

SELECT *
FROM class
LEFT JOIN class_info
ON class.id = class_info.id
WHERE class_info.id IS NULL
UNION
SELECT *
FROM class
RIGHT JOIN class_info
ON class.id = class_info.id
WHERE class.id IS NULL;

```

```

105
106 SELECT *
107 FROM class
108 LEFT JOIN class_info
109 ON class.id = class_info.id
110 WHERE class_info.id IS NULL
111 UNION
112 SELECT *
113 FROM class
114 RIGHT JOIN class_info
115 ON class.id = class_info.id
116 WHERE class.id IS NULL;
117
100% 1:104

```

Result Grid

id	name	id	address
4	Anusri	NULL	NULL
5	Rohan	NULL	NULL
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

Result Grid Form Editor

Step 17 — Create first_table and second_table (for UNION section)

Query:

```
CREATE TABLE first_table(  
  id INT,  
  name VARCHAR(30)  
);  
  
CREATE TABLE second_table(  
  id INT,  
  name VARCHAR(30)  
);  
  
INSERT INTO first_table VALUES  
(1,'Hansika'),  
(2,'Hasini');  
  
INSERT INTO second_table VALUES  
(2,'Hasini'),  
(3,'Rohan');
```

The screenshot shows a database IDE interface. On the left, there's a sidebar with icons for 'Status and Data Export', 'Data Export', and 'Data Import'. Below these is a tabbed interface with 'Object Explorer' and 'Session'. The 'Object Explorer' tab is active, showing 'No object selected'. The main area displays a SQL script with line numbers 119 to 137. The script contains the same SQL code as in the previous block. The 'Session' tab is also visible, showing 'Action Output'. At the bottom, there's a table with columns: 'Time', 'Action', 'Response', and 'Duration / Fetch Time'. The table contains one row with a green checkmark icon, the number 82, the time 10:46:33, the action 'INSERT INTO...', and the response '2 row(s) affected Records: 2 Duplicates: 0 Warnings...'. Below the table, it says 'Query Completed'.

```
119  
120 CREATE TABLE first_table(  
121     id INT,  
122     name VARCHAR(30)  
123 );  
124  
125 CREATE TABLE second_table(  
126     id INT,  
127     name VARCHAR(30)  
128 );  
129  
130 INSERT INTO first_table VALUES  
131     (1,'Hansika'),  
132     (2,'Hasini');  
133  
134 INSERT INTO second_table VALUES  
135     (2,'Hasini'),  
136     (3,'Rohan');  
137
```

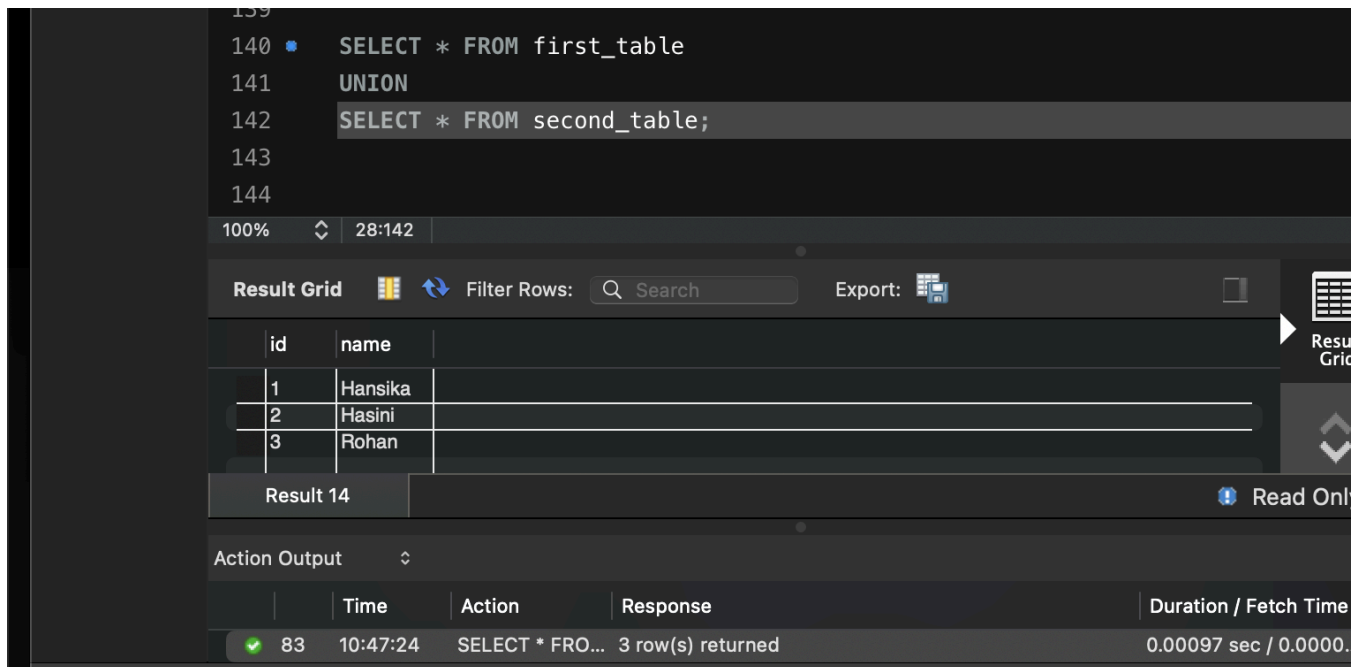
	Time	Action	Response	Duration / Fetch Time
82	10:46:33	INSERT INTO...	2 row(s) affected Records: 2 Duplicates: 0 Warnings...	0.0012 sec

Query Completed

Step 18 — Question 11: UNION

Query:

```
SELECT * FROM first_table
UNION
SELECT * FROM second_table;
```



139

```
140 SELECT * FROM first_table
141 UNION
142 SELECT * FROM second_table;
143
144
```

100% 28:142

Result Grid Filter Rows: Search Export:

id	name
1	Hansika
2	Hasini
3	Rohan

Result 14 Read Only

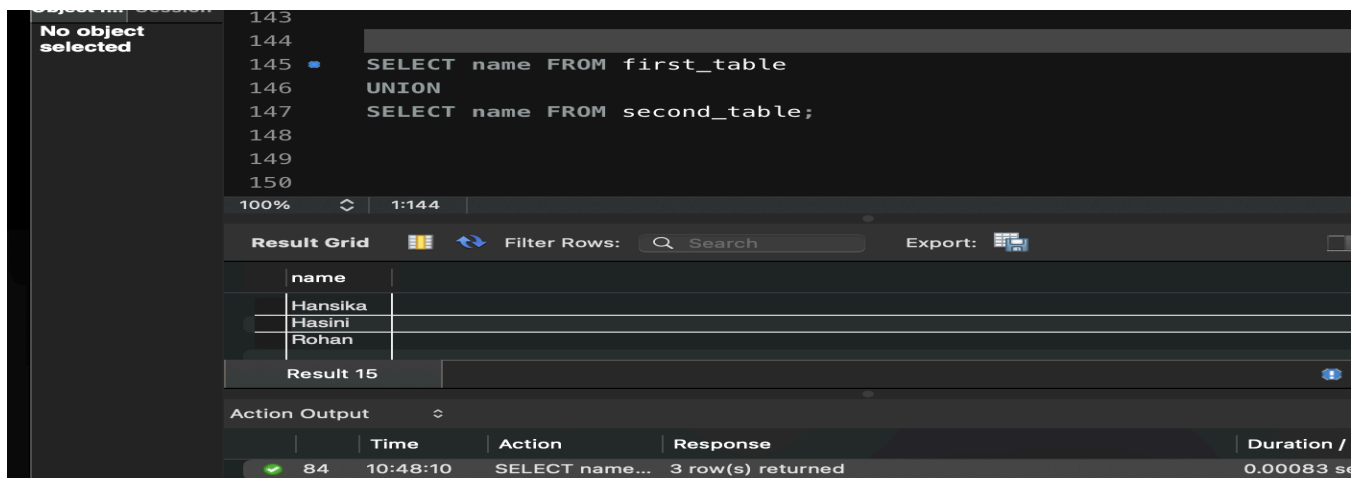
Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 83	10:47:24	SELECT * FRO...	3 row(s) returned	0.00097 sec / 0.0000.

Step 19 — Question 12: UNION (Names Only)

Query:

```
SELECT name FROM first_table
UNION
SELECT name FROM second_table;
```



No object selected

```
143
144
145 SELECT name FROM first_table
146 UNION
147 SELECT name FROM second_table;
148
149
150
```

100% 1:144

Result Grid Filter Rows: Search Export:

name
Hansika
Hasini
Rohan

Result 15

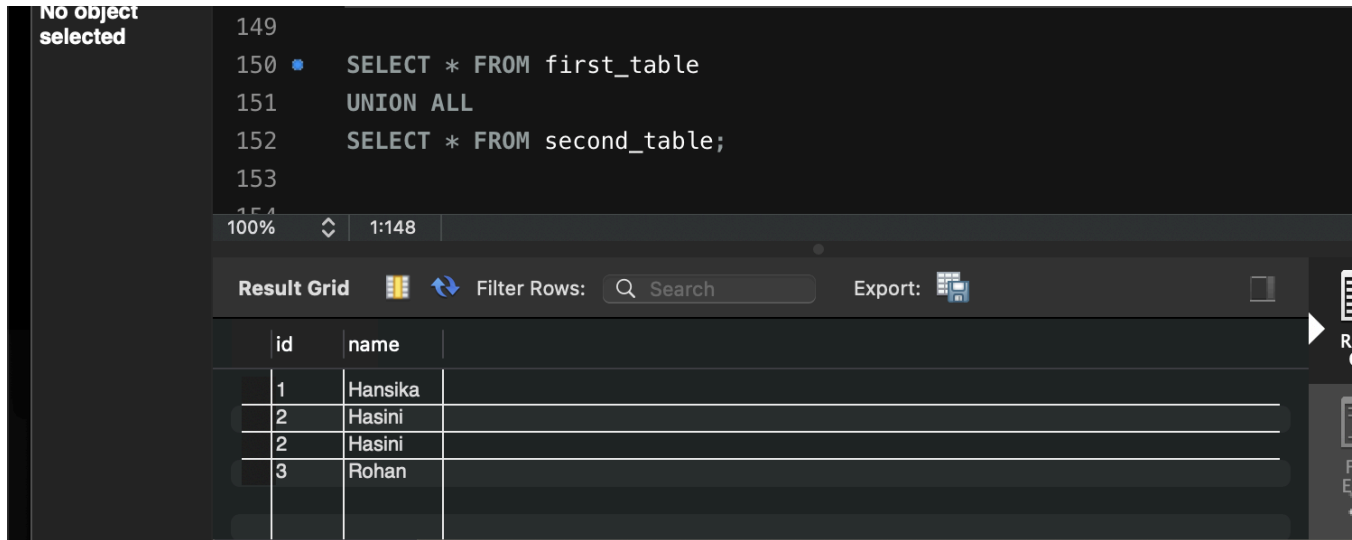
Action Output

	Time	Action	Response	Duration /
✓ 84	10:48:10	SELECT name...	3 row(s) returned	0.00083 se

Step 20 — Question 13: UNION ALL

Query:

```
SELECT * FROM first_table
UNION ALL
SELECT * FROM second_table;
```



No object selected

```
149
150 • SELECT * FROM first_table
151     UNION ALL
152     SELECT * FROM second_table;
153
```

100% 1:148

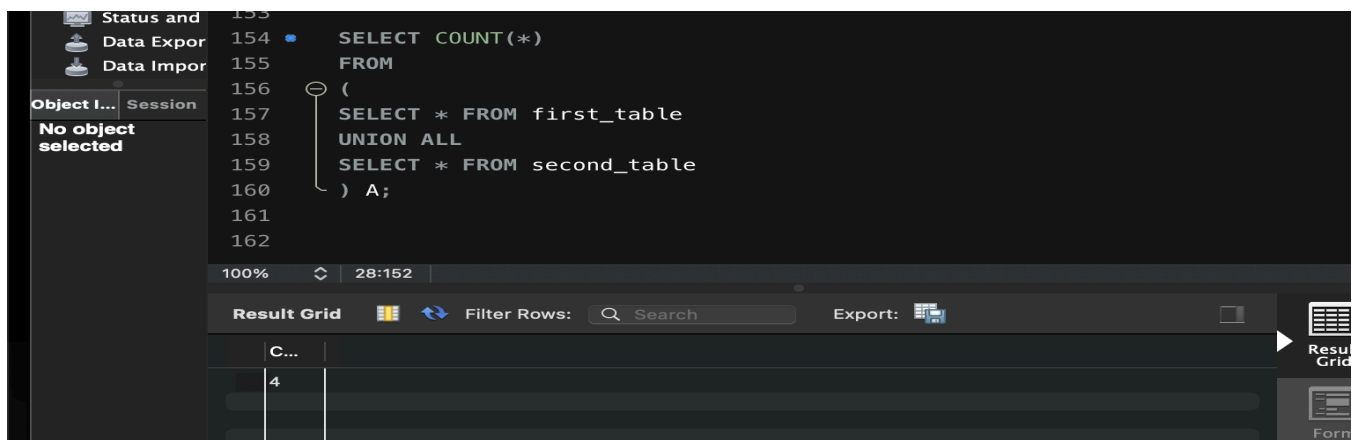
Result Grid Filter Rows: Search Export:

	id	name
	1	Hansika
	2	Hasini
	2	Hasini
	3	Rohan

Step 21 — Question 14: UNION ALL (Row Count)

Query:

```
SELECT COUNT(*)
FROM
(
SELECT * FROM first_table
UNION ALL
SELECT * FROM second_table
) A;
```



Status and Data Export Data Import

Object Explorer Session

No object selected

```
153
154 • SELECT COUNT(*)
155     FROM
156     (
157     SELECT * FROM first_table
158     UNION ALL
159     SELECT * FROM second_table
160     ) A;
161
162
```

100% 28:152

Result Grid Filter Rows: Search Export:

C...
4

Step 22 — Question 15: INTERSECT

Query:

```
SELECT * FROM first_table
INTERSECT
SELECT * FROM second_table;

-- Alternative if INTERSECT is not supported:
SELECT f.* FROM first_table f
INNER JOIN second_table s
ON f.id = s.id AND f.name = s.name;
```

The screenshot shows a SQL IDE interface. On the left, a sidebar indicates "No object selected". The main editor displays the following SQL query:

```
164
165 -- Alternative if INTERSECT is not supported:
166 SELECT f.* FROM first_table f
167 INNER JOIN second_table s
168 ON f.id = s.id AND f.name = s.name;
169
```

Below the editor, the "Result Grid" is visible, showing a single row of results:

id	name
2	Hasini

The interface also includes a search bar, an "Export" button, and a status bar showing "1:164".

Step 23 — Question 16: INTERSECT (Names Only)

Query:

```
SELECT name FROM first_table
INTERSECT
SELECT name FROM second_table;

-- Alternative if INTERSECT is not supported:
SELECT name FROM first_table
WHERE name IN (SELECT name FROM second_table);
```

The screenshot shows a SQL IDE interface. The main editor displays the following SQL query:

```
177
178 -- Alternative if INTERSECT is not supported:
179 SELECT name FROM first_table
180 WHERE name IN (SELECT name FROM second_table);
181
```

Below the editor, the "Result Grid" is visible, showing a single row of results:

name
Hasini

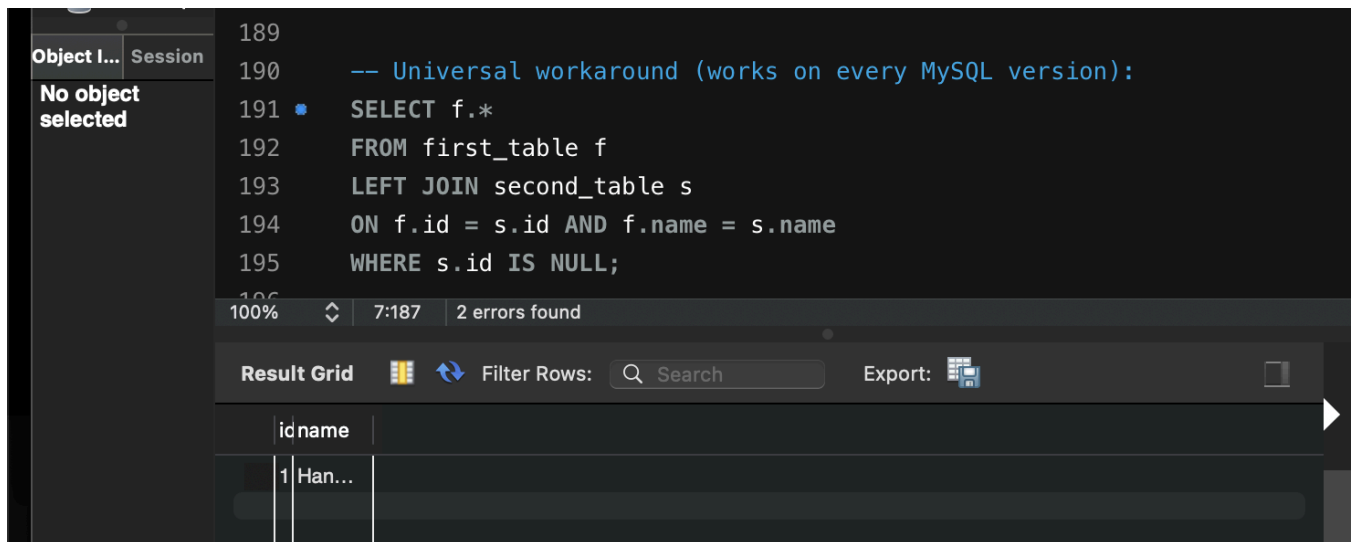
The interface also includes a search bar, an "Export" button, and a status bar showing "31:176 | 1 error found".

Step 24 — Question 17: MINUS

Query:

```
-- If your MySQL version supports EXCEPT:
SELECT * FROM first_table
EXCEPT
SELECT * FROM second_table;

-- Universal workaround (works on every MySQL version):
SELECT f.*
FROM first_table f
LEFT JOIN second_table s
ON f.id = s.id AND f.name = s.name
WHERE s.id IS NULL;
```

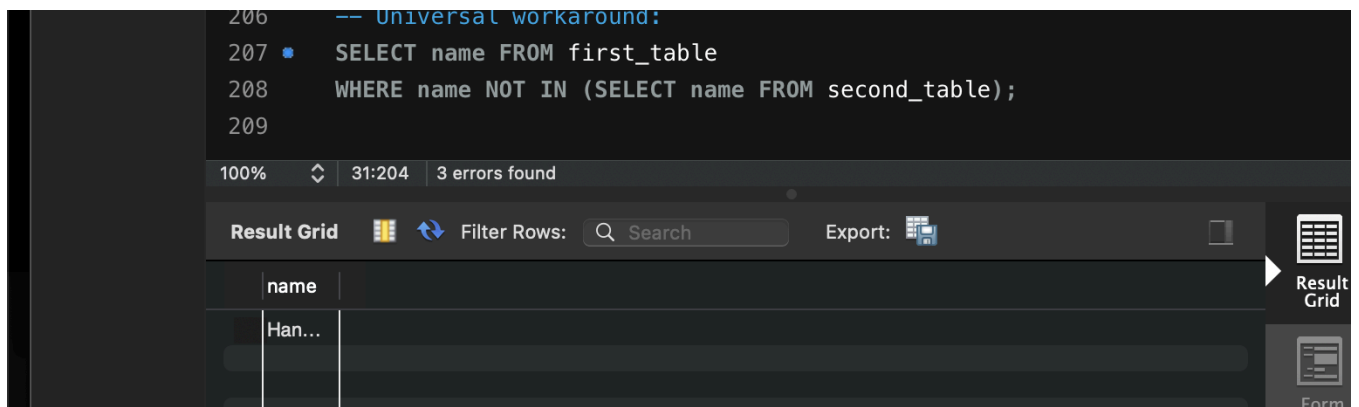


Step 25 — Question 18: MINUS (Names Only)

Query:

```
-- If your MySQL version supports EXCEPT:
SELECT name FROM first_table
EXCEPT
SELECT name FROM second_table;

-- Universal workaround:
SELECT name FROM first_table
WHERE name NOT IN (SELECT name FROM second_table);
```

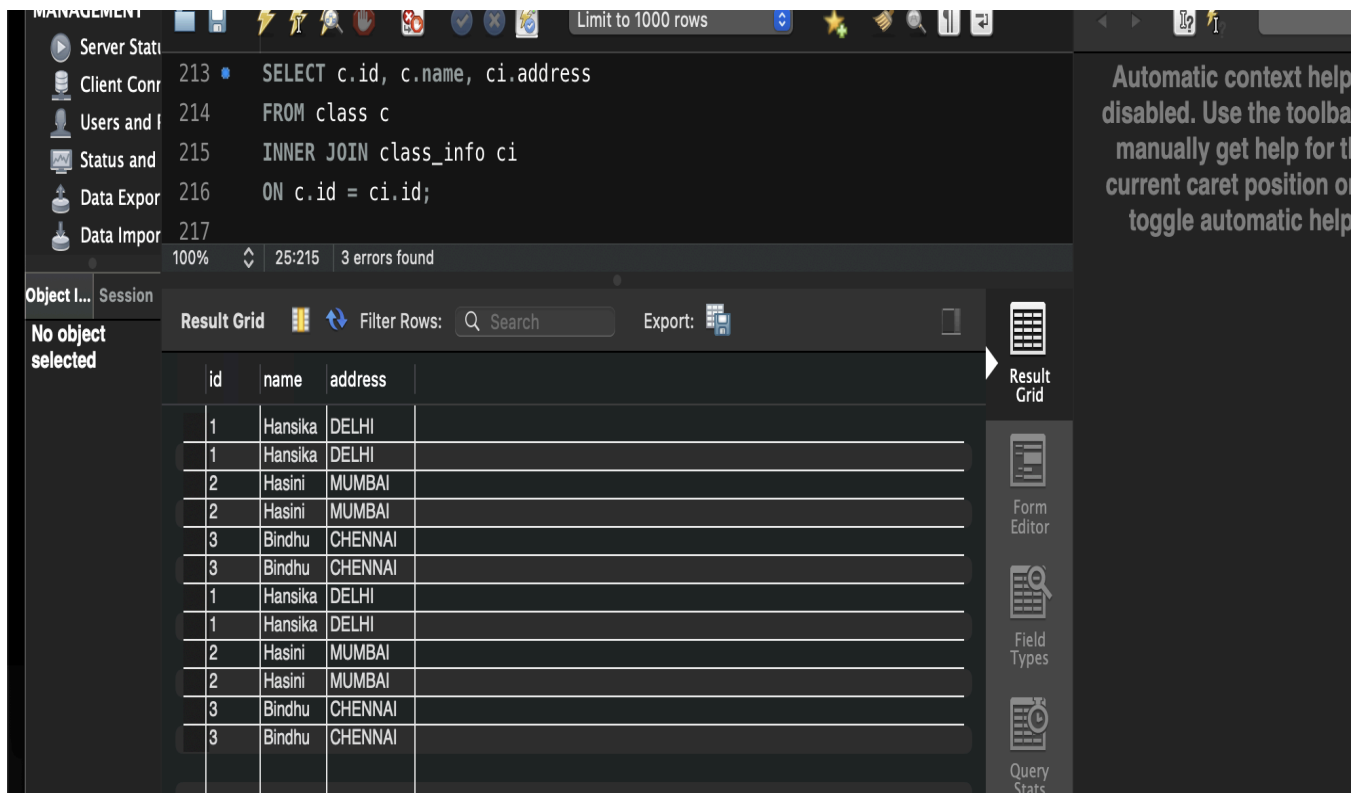
Step 26 — Question 19: Advanced — Matching Addresses

Query:

```

SELECT c.id, c.name, ci.address
FROM class c
INNER JOIN class_info ci
ON c.id = ci.id;

```



Step 27 — Question 20: Advanced — Address Availability Report

Query:

```
SELECT c.id,  
       c.name,  
       CASE  
         WHEN ci.address IS NULL  
         THEN 'Address Missing'  
         ELSE 'Address Available'  
       END AS Status  
FROM class c  
LEFT JOIN class_info ci  
ON c.id = ci.id;
```

	id	name	Status	
	1	Hansika	Address Available	
	1	Hansika	Address Available	
	2	Hasini	Address Available	
	2	Hasini	Address Available	
	3	Bindhu	Address Available	
	3	Bindhu	Address Available	
	4	Anusri	Address Missing	
	1	Hansika	Address Available	
	1	Hansika	Address Available	
	2	Hasini	Address Available	
	2	Hasini	Address Available	
	3	Bindhu	Address Available	
	3	Bindhu	Address Available	
	4	Anusri	Address Missing	
	5	Rohan	Address Missing	

Result Grid
Form Editor
Field Types
Query Stats